

P0330R4

Literal Suffixes for ptrdiff_t and size_t

Published Proposal, 2019-01-21

Authors:

[JeanHeyd Meneide](#)
Rein Halbersma

Audience:

EWG, CWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Latest:

https://theohd.github.io/vendor/future_cxx/papers/d0330.html

Abstract

This paper proposes core language suffixes for `size_t` and `ptrdiff_t`.

Table of Contents

1	Revision History
1.1	Revision 4 - January 21st, 2019
1.2	Revision 3 - November 26th, 2018
1.3	Revision 2 - October 1st, 2018
1.4	Revision 1 - October 12th, 2017
1.5	Revision 0 - November 21st, 2014
2	Motivation
3	Design
3.1	But what about using <code>t</code> for <code>ptrdiff_t</code> and <code>zu</code> for <code>size_t</code> instead?
3.2	Why bother making a suffix for <code>ptrdiff_t</code>
3.3	What about the fixed/least/max (unsigned) int types?
4	Impact on the Standard
5	Proposed wording and Feature Test Macros
5.1	Proposed feature Test Macro
5.2	Intent
5.3	Proposed Wording
6	Acknowledgements
7	Feedback on Revisions
	References
	Informative References

§ 1. Revision History

§ 1.1. Revision 4 - January 21st, 2019

- Discussed additional reasoning due to `<span>` changes with `ssize(const T&)` and others.
- Wording is now relative to [\[n4778\]](#), the latest C++ Standard working draft.

§ 1.2. Revision 3 - November 26th, 2018

- Strengthened rationale for the current set of suffixes in [§3.1 But what about using t for ptrdiff_t and zu for size_t instead?](#).

§ 1.3. Revision 2 - October 1st, 2018

- Published as P0330R2; change reply-to and point of contact from Rein Halbersma to JeanHeyd Meneide, who revitalized paper according to feedback from Rein Halbersma, and all of LWG. Overwhelming consensus for it to be a Language Feature instead, proposal rewritten as Language proposal with wording against [\[n4762\]](#).

§ 1.4. Revision 1 - October 12th, 2017

- Published as P0330R1; expanded the survey of existing literals. Synced the proposed wording with the Working Draft WG21/N4687. Moved the reference implementation from BitBucket to GitHub.

§ 1.5. Revision 0 - November 21st, 2014

- Initial release; published as N4254.
- Published as P0330R0; summarized LEWG's view re N4254; dropped the proposed suffix for `ptrdiff_t`; changed the proposed suffix for `size_t` to `zu`; added survey of existing literal suffixes.

§ 2. Motivation

Currently	With Proposal
<pre>std::vector<int> v{0, 1, 2, 3}; for (auto i = 0u, s = v.size(); i < s; ++i) { /* use both i and v[i] */ }</pre> ⚠ - Compiles on 32-bit, truncates (maybe with warnings) on 64-bit <pre>std::vector<int> v{0, 1, 2, 3}; for (auto i = 0, s = v.size(); i < s; ++i) { /* use both i and v[i] */ }</pre> ✗ - Compilation error	<pre>std::vector<int> v{0, 1, 2, 3}; for (auto i = 0uz, s = v.size(); i < s; ++i) { /* use both i and v[i] */ }</pre> ✓ - Compiles with no warnings on 32-bit or 64-bit

Consider this very simple code to print an index and its value:

```
std::vector<int> v{0, 1, 2, 3};
for (auto i = 0; i < v.size(); ++i) {
    std::cout << i << ": " << v[i] << '\n';
}
```

This code can lead to the following warnings:

```
main.cpp: In function 'int main()':
main.cpp:warning: comparison of integer expressions
of different signedness: 'int' and 'long unsigned int' [-Wsign-compare]
    for (auto i = 0; i < v.size(); ++i) {
        ~~~~~^~~~~~
```

It grows worse if a user wants to cache the size rather than query it per-iteration:

```
std::vector<int> v{0, 1, 2, 3};
for (auto i = 0, s = v.size(); i < s; ++i) {
    /* use both i and v[i] */
}
```

Resulting in a hard compiler error:

```
main.cpp: In function 'int main()':
main.cpp:8:10: error: inconsistent deduction
for 'auto': 'int' and then 'long unsigned int'
    for (auto i = 0, s = v.size(); i < s; ++i) {
        ^~~~~
```

This paper proposes adding a `zu` literal suffix that deduces literals to `size_t`, making the following warning-free:

```
for (auto i = 0zu; i < v.size(); ++i) {
    std::cout << i << ": " << v[i] << '\n';
}
```

More generally:

- `int` is the default type deduced from integer literals without suffix;
- comparisons, signs or conversion ranks with integers can lead to surprising results;
- `size_t` -- and less frequently, `ptrdiff_t` -- are nearly impossible to avoid in the standard library for element access or `.size()` members;
- programmer intent and stability is hard to communicate portably with the current set of literals;
- surprises range from (pedantic) compiler errors to undefined behavior;
- existing integer suffixes (such as `ul`) are not a general solution, e.g. when switching compilation between 32-bit and 64-bit on common architectures;
- and, C-style casts and `static_casts` are verbose.

§ 3. Design

Following the feedback from [§7 Feedback on Revisions](#), we have dropped the `std::support_literals` User-Defined Literals and chose a Core Language Literal Suffix. We opine that it would better serve the needs of addressing the motivation.

As a language feature, the design of the suffixes becomes much simpler. The core language only has one format for its integer literal suffixes: the letter(s), with an optional `u` on either side of the letter(s) to make it unsigned, with the signed variant being the default on most architectures. This ruled out using `s` and `sz`, because that would produce an inconsistent set of suffixes with the rest of the language.

The literal suffixes `z` and `uz/zu` were chosen to represent signed and unsigned, respectively. `decltype(0z)` will yield `ptrdiff_t` and `decltype(0uz)/decltype(0zu)` will yield `size_t`. Like other case-insensitive language literal suffixes, it will accept both `Z` and `z` (and `U` and `u` alongside of it). This follows the current convention of the core language to be able to place `u` and `z` in any order / any case for the suffix.

§ 3.1. But what about using `t` for `ptrdiff_t` and `zu` for `size_t` instead?

Consider the following snippet:

```
int main() {
    signed decltype(sizeof(0)) x = 0;
    unsigned decltype((char*)nullptr - (char*)nullptr) y = 0;
    return x - y;
}
```

When compiled, these warnings appear:

```
main.cpp:2:32: warning: long, short,
signed or unsigned used invalidly for 'x' [-Wpedantic]
main.cpp:3:56: warning: long, short,
signed or unsigned used invalidly for 'y' [-Wpedantic]
```

These warnings are, in fact, correct: `signed size_t` and `unsigned ptrdiff_t` do not exist in the C++ standard. The POSIX standard defines just `ssize_t` to be `signed size_t`, but it does not define `unsigned ptrdiff_t`. In order to keep parity with the Core Language's consistency, one would need to provide `ut` and `z` counterparts, for which there is no existing definition in the C++ standard. This makes providing 2 separate suffixes a poor idea unless someone is willing to pin these fundamental types down.

This paper is not attempting to make such a definition.

§ 3.2. Why bother making a suffix for `ptrdiff_t`

With the inclusion of a `ssize()` free function coming to the standard, this paper advocates for keeping a literal for `ptrdiff_t`. As the paper was going through the Library group earlier, `span`'s design decisions were not coming to a head and thusly the dialogue did not bring this up. With `span` now headed into C++20 and `ssize()` with it, having a modifier for `ptrdiff_t` is useful for consistency and helpful for success in a world where developers employ a lot of `auto` and `decltype`.

§ 3.3. What about the fixed/least/max (unsigned) int types?

This paper does not propose suffixes for the fixed size, at-least size, and max size integral types in the standard library or the language. This paper is focusing exclusively on `ptrdiff_t` and `size_t`. We have also been made aware of another paper which may handle this separately and considers all the design space necessary for such.

§ 4. Impact on the Standard

This feature is purely an extension of the language and has, to the best of our knowledge, no conflict with existing or currently proposed features. `z` is currently not a literal suffix in the language. As a proof of concept, it has a [patch in GCC already](#) according to this paper by Ed Smith-Rowland.

§ 5. Proposed wording and Feature Test Macros

The following wording is relative to [\[n4778\]](#).

§ 5.1. Proposed feature Test Macro

The recommended feature test macro is `__cpp_ptrdiff_t_suffix`.

§ 5.2. Intent

The intent of this paper is to propose 2 language suffixes for integral literals of specific types. One is for `ptrdiff_t`, one is for `size_t`. We follow the conventions set out for other literals in the standard. We define the suffix to produce types `size_t` and `ptrdiff_t` similar to how [§5.13.7 Pointer Literals \[lex.nullptr\]](#) introduces `std::nullptr_t`.

§ 5.3. Proposed Wording

Modify §5.13.2 Integer Literals [**lex.icon**] with additional suffixes:

integer-suffix:

unsigned-suffix long-suffix_{opt}

unsigned-suffix long-long-suffix_{opt}

unsigned-suffix ptrdiff-suffix_{opt}

long-suffix unsigned-suffix_{opt}

long-long-suffix unsigned-suffix_{opt}

ptrdiff-suffix unsigned-suffix_{opt}

unsigned-suffix: one of

u U

long-suffix: one of

l L

long-long-suffix: one of

ll LL

ptrdiff-suffix: one of

z Z

Append to §5.13.2 Integer Literals [**lex.icon**]'s **Table 7** two additional entries:

Suffix	Decimal literal	Binary, octal, or hexadecimal literal
<u>z or Z</u>	<u>ptrdiff t</u>	<u>ptrdiff t</u>
<u>Both u or U</u> <u>and z or Z</u>	<u>size t</u>	<u>size t</u>

Append to §14.8 Predefined macro names [**cpp.predefined**]'s **Table 16** with one additional entry:

Macro name	Value
<u>cpp_ptrdiff_t_suffix</u>	<u>201902L</u>

§ 6. Acknowledgements

Thank you to Rein Halbersma, who started this paper and put in the necessary work for r0 and r1. Thank you to Walter E. Brown, who acted as *locum* on this paper before the Committee twice and gave us valuable feedback on wording. Thank you to Lounge<C++>'s Cicada for encouraging us to write this paper. Thank you to Hubert Tong for giving us a few pointers on where in the Core Language to modify things for such a paper. Thank you to Tim Song for wording advice.

We appreciate your guidance as we learn to be a better Committee member and represent the C++ community's needs more more efficiently and effectively in the coming months.

§ 7. Feedback on Revisions

Polls are in the form **Strongly in Favor** | **Favor** | **Neutral** | **Against** | **Strongly Against**. The polls on Revision 1 were as follows, from LWG at the WG21 Albuquerque meeting.

Proposal as presented, i.e., are we OK with the library solution going forward?

0 | 6 | 5 | 7 | 4

We translated this as strong discouragement to pursue this feature as a set of user-defined literals. A second poll was taken.

Do we want to solve this problem with a language feature?

2 | 15 | 0 | 2 | 2

We considered this overwhelming consensus for it to be a language feature instead, culminating in this paper after much feedback.

§ References

§ Informative References

[GCC-IMPLEMENTATION]

Ed Smith-Rowland. [\[\[C++ PATCH\]\] Implement C++2a P0330R2 - Literal Suffixes for ptrdiff_t and size_t](https://gcc.gnu.org/ml/gcc-patches/2018-10/msg01278.html). October 21st, 2018. URL: <https://gcc.gnu.org/ml/gcc-patches/2018-10/msg01278.html>

[N4762]

ISO/IEC JTC1/SC22/WG21 - The C++ Standards Committee; Richard Smith. [N4762- Working Draft, Standard for Programming Language C++](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4762.pdf). May 10th, 2018. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4762.pdf>

[N4778]

ISO/IEC JTC1/SC22/WG21 - The C++ Standards Committee; Richard Smith. [N4778 - Working Draft, Standard for Programming Language C++](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4778.pdf). November 26th, 2018. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4778.pdf>